

PostgreSQL Stored Procedures — Line-by-Line Notes

Beginner study PDF based on the supplied SQL. Each important line is explained in simple language, followed by the example code.

1. Basic Procedure Syntax

- `CREATE PROCEDURE` → tells PostgreSQL to create a stored procedure.
- `procedure_name()` → the name of the procedure. Empty parentheses mean there are no parameters.
- `LANGUAGE plpgsql` → says the procedure is written in PostgreSQL's procedural language, PL/pgSQL.
- `AS $$` → starts the procedure body. `$$` is dollar quoting.
- `BEGIN` → starts the executable statements.
- `-- SQL statements` → the actual SQL or PL/pgSQL logic goes here.
- `END;` → closes the executable block.
- `$$;` → closes the procedure definition.

```
CREATE PROCEDURE procedure_name()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
-- SQL statements  
END;  
$$;
```

2. First Stored Procedure

- `RAISE NOTICE` → displays an informational message.
- `CALL hello_world();` → executes the procedure.
- Expected message: Hello World.

```
CREATE PROCEDURE hello_world()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
RAISE NOTICE 'Hello World';  
END;  
$$;
```

```
CALL hello_world();
```

3. Procedure Without Parameters

- No parameters are supplied.
- `CALL` executes the procedure.
- `RAISE NOTICE` shows the success message.

```
CREATE PROCEDURE test_message()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
RAISE NOTICE 'Procedure Executed Successfully';  
END;  
$$;
```

```
CALL test_message();
```

4. Procedure With Parameters — INSERT

- CREATE TABLE → creates the table used by the procedure.
- p_id, p_name, p_salary → input parameters.
- INSERT INTO employees1(...) → identifies the columns receiving data.
- VALUES (p_id, p_name, p_salary) → puts the procedure parameter values into the table.
- CALL add_employee(...) → supplies actual values to the parameters.
- SELECT * → displays the inserted rows.

```
CREATE TABLE employees1 (  
  employee_id INT,  
  employee_name VARCHAR(100),  
  salary NUMERIC  
);  
  
CREATE PROCEDURE add_employee(  
  p_id INT,  
  p_name VARCHAR,  
  p_salary NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  INSERT INTO employees1(employee_id, employee_name, salary)  
  VALUES (p_id, p_name, p_salary);  
END;  
$$;  
  
CALL add_employee(101, 'Rahul', 50000);  
CALL add_employee(102, 'Kriya', 10000);  
CALL add_employee(103, 'Jenish', 100000);  
  
SELECT * FROM employees1;
```

5. UPDATE Procedure

- p_id → identifies the employee.
- p_amount → amount to add to salary.
- UPDATE → changes the employee's salary.
- SET salary = salary + p_amount → increases the existing salary.
- WHERE employee_id = p_id → updates only the selected employee.

```
CREATE PROCEDURE increase_salary (  
  p_id INT,  
  p_amount NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  UPDATE employees1  
  SET salary = salary + p_amount  
  WHERE employee_id = p_id;  
END;  
$$;  
  
CALL increase_salary(103, 500000);
```

```
SELECT * FROM employees1;
```

6. DELETE Procedure

- DELETE FROM → removes rows.
- WHERE employee_id = p_id → prevents deleting every employee.
- CALL delete_employee(101) → deletes employee 101.

```
CREATE PROCEDURE delete_employee(  
  p_id INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  DELETE FROM employees1  
  WHERE employee_id = p_id;  
END;  
$$;
```

```
CALL delete_employee(101);
```

```
SELECT * FROM employees1;
```

7. IF / ELSE

- IF → starts a condition.
- p_salary >= 50000 → condition being tested.
- THEN → starts the code executed when the condition is true.
- ELSE → runs when the condition is false.
- END IF; → closes the IF statement.

```
CREATE PROCEDURE check_salary(  
  p_salary NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  IF p_salary >= 50000 THEN  
    RAISE NOTICE 'Good Salary';  
  ELSE  
    RAISE NOTICE 'Under 50000';  
  END IF;  
END;  
$$;
```

```
CALL check_salary(30000);
```

8. IF / ELSIF / ELSE

- CREATE OR REPLACE → creates the procedure or replaces its existing definition.
- IF → tests the first condition.
- ELSIF → tests another condition if the previous condition was false.
- ELSE → handles all remaining cases.

- Only the first true branch is executed.

```
CREATE OR REPLACE PROCEDURE salary_category(
p_salary NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
IF p_salary >= 100000 THEN
RAISE NOTICE 'High Salary';
ELSIF p_salary >= 50000 THEN
RAISE NOTICE 'Medium Salary';
ELSE
RAISE NOTICE 'Low Salary';
END IF;
END;
$$;

CALL salary_category(1000000);
```

9. Variables

- DECLARE → starts the variable declaration section.
- variable_name → the name of the variable.
- datatype → the variable's data type.
- Variables are normally declared before BEGIN.

```
DECLARE
variable_name datatype;
```

10. Variable Example

- DECLARE → begins variable declarations.
- total NUMERIC → creates a numeric variable.
- total = 100 + 200 → calculates and stores the total. In PL/pgSQL, := is the recommended assignment operator.
- RAISE NOTICE 'Total = %', total → % is replaced by the variable value.
- CALL → runs the procedure.

```
CREATE PROCEDURE calculate_total ()
LANGUAGE plpgsql
AS $$
DECLARE
total NUMERIC;
BEGIN
total = 100 + 200;
RAISE NOTICE 'Total = %', total;
END;
$$;

CALL calculate_total();
```

11. SELECT INTO

- v_salary → local variable for storing the salary.
- SELECT salary INTO v_salary → gets the selected salary and stores it in v_salary.
- WHERE employee_id = p_id → searches for the requested employee.

- IF v_salary IS NULL → checks whether no usable salary value was obtained.
- ELSE → displays the salary when a value exists.

```
CREATE PROCEDURE get_employee_salary(
p_id INT
)
LANGUAGE plpgsql
AS $$
DECLARE
v_salary NUMERIC;
BEGIN
SELECT salary
INTO v_salary
FROM employees1
WHERE employee_id = p_id;

IF v_salary IS NULL THEN
RAISE NOTICE 'Employee % not found', p_id;
ELSE
RAISE NOTICE 'Salary = %', v_salary;
END IF;
END;
$$;

CALL get_employee_salary(101);
```

12. COUNT

- COUNT(*) → counts rows.
- INTO v_count → stores the count in the variable.
- RAISE NOTICE → displays the number of employees.

```
CREATE PROCEDURE employee_count()
LANGUAGE plpgsql
AS $$
DECLARE
v_count INT;
BEGIN
SELECT COUNT(*)
INTO v_count
FROM employees1;

RAISE NOTICE 'Total Employees = %', v_count;
END;
$$;

CALL employee_count();
```

13. LOOP

- LOOP → starts a repeated block.
- v_numbers = 1 → initial value. For PL/pgSQL, := is recommended for assignment.
- v_numbers = v_numbers + 1 → increases the number by one.
- EXIT WHEN v_numbers > 100 → stops the loop after reaching 100.
- END LOOP → closes the loop.

```
CREATE PROCEDURE print_numbers()
LANGUAGE plpgsql
AS $$
DECLARE
v_numbers INT = 1;
```

```

BEGIN
LOOP
RAISE NOTICE 'Number = %', v_numbers;
v_numbers = v_numbers + 1;
EXIT WHEN v_numbers > 100;
END LOOP;
END;
$$;

CALL print_numbers();

```

14. Parameter Modes — IN / OUT / INOUT

- IN → input value enters the procedure.
- OUT → procedure returns a value through the parameter.
- INOUT → parameter enters with a value and can be changed and returned.

```

IN → Give data
OUT → Get data
INOUT → Give + get data

```

15. IN Parameters

- IN explicitly marks each parameter as an input parameter.
- The procedure uses those values in the INSERT statement.
- Note: employees5 must exist before this procedure is called.

```

CREATE PROCEDURE add_employee5(
IN p_id INT,
IN p_name VARCHAR,
IN p_salary NUMERIC
)
LANGUAGE plpgsql
AS $$
BEGIN
INSERT INTO employees5
VALUES (p_id, p_name, p_salary);
END;
$$;

```

16. OUT Parameter

- OUT total_employees → creates an output parameter.
- SELECT COUNT(*) INTO total_employees → puts the count into the output parameter.
- Important: the supplied table name is employees, so that table must exist.
- CALL ... → executes the procedure and PostgreSQL returns the OUT value.

```

CREATE PROCEDURE get_employee_count(
OUT total_employees INT
)
LANGUAGE plpgsql
AS $$
BEGIN
SELECT COUNT(*)
INTO total_employees
FROM employees;
END;
$$;

```

```
CALL get_employee_count(NULL);
```

17. INOUT Parameter

- INOUT → accepts an input and returns the changed value.
- `p_amount = p_amount + 100` → adds 100.
- Calling with 500 produces 600.
- For PL/pgSQL style, `:=` is recommended for assignment.

```
CREATE PROCEDURE increase_amount(  
  INOUT p_amount INT  
)  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  p_amount = p_amount + 100;  
END;  
$$;
```

```
CALL increase_amount(500);
```

18. DROP PROCEDURE

- DROP PROCEDURE → permanently removes the procedure definition.
- If the procedure has parameters, specify their data types when identifying the procedure.

```
DROP PROCEDURE procedure_name();
```

19. Procedure Calling Another Procedure

- `procedure_a` → first procedure.
- `procedure_b` → second procedure.
- `CALL procedure_a()` → `procedure_b` executes `procedure_a`.
- `CALL procedure_b()` → starts the chain.
- Expected messages: Procedure A, then Procedure B.

```
CREATE PROCEDURE procedure_a()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  RAISE NOTICE 'Procedure A';  
END;  
$$;
```

```
CREATE PROCEDURE procedure_b()  
LANGUAGE plpgsql  
AS $$  
BEGIN  
  CALL procedure_a();  
  RAISE NOTICE 'Procedure B';  
END;  
$$;
```

```
CALL procedure_b();
```

20. Mini Project — Bank Withdrawal

- bank_account → stores account information.
- PRIMARY KEY → uniquely identifies each account.
- v_balance → stores the current balance.
- SELECT balance INTO v_balance → reads the account balance.
- IF v_balance >= p_amount → checks whether enough money exists.
- UPDATE → subtracts the withdrawal amount.
- RAISE NOTICE → tells the user whether the withdrawal succeeded.
- CALL withdraw_money(1, 3000) → withdraws 3000 from account 1.
- Final SELECT → displays the updated account balances.

```
CREATE TABLE bank_account (  
  account_id INT PRIMARY KEY,  
  account_holder VARCHAR(100),  
  balance NUMERIC  
);  
  
INSERT INTO bank_account  
VALUES  
(1, 'Rahul', 10000),  
(2, 'Amit', 5000);  
  
CREATE PROCEDURE withdraw_money(  
  p_account_id INT,  
  p_amount NUMERIC  
)  
LANGUAGE plpgsql  
AS $$  
DECLARE  
  v_balance NUMERIC;  
BEGIN  
  SELECT balance  
  INTO v_balance  
  FROM bank_account  
  WHERE account_id = p_account_id;  
  
  IF v_balance >= p_amount THEN  
    UPDATE bank_account  
    SET balance = balance - p_amount  
    WHERE account_id = p_account_id;  
  
    RAISE NOTICE 'Withdrawal successful';  
  ELSE  
    RAISE NOTICE 'Insufficient balance';  
  END IF;  
END;  
$$;  
  
CALL withdraw_money(1, 3000);  
  
SELECT * FROM bank_account;
```

Important Corrections in the Supplied Code

- Use := for PL/pgSQL variable assignment, for example: total := 100 + 200; and v_numbers := v_numbers + 1;.
- employees5 must be created before add_employee5 is called.

- The OUT example uses the employees table, while the earlier examples created employees1. Make sure the referenced table exists.
- get_employee_salary(101) may report NULL if employee 101 was already deleted by delete_employee(101). Run the examples in the correct order or use an existing employee ID.
- The loop prints 1 through 100 because it exits after incrementing from 100 to 101.